SHRSS – AEM Assets API Guide - Classic

1 Create AEM technical account & service credentials

1.1 Prerequisites (roles & access)

Before starting, confirm the following:

1. **Adobe IMS Org role**
    - The person doing the setup must be an **IMS Org System Administrator** for the SHRSS org.
2. **AEM product profile**
    - That same user must be a member of an AEM **Author** product profile with admin-level access, for example:
        - AEM Administrators - author - Program <ID> - Environment <ID>
    - This is required to:
        - Open **Cloud Manager** for the correct program
        - Access the **AEM Developer Console**
        - Create technical accounts & download service credentials
3. **Cloud Manager access**
    - User can log into:
        - https://experience.adobe.com/#/cloud-manager
    - And see the SHRSS AEM program and environments (Dev/Stage/Prod).

If any of the above is missing, skip to **Appendix A – Support ticket template**.

1.2 Create Technical Account

These steps are done **once per AEM environment** (e.g., once for Prod author, optionally again for Stage/Dev).

Important – JWT deprecation vs AEM Developer Console

- **Adobe Developer Console – Service Account (JWT)** credentials are deprecated and must be migrated to **OAuth Server-to-Server** credentials. See *"JWT Credentials Deprecation in Adobe Developer Console"* for details.
- This guide's "classic" HTTP API examples use **AEM Developer Console service credentials (technical account)**, which internally generate a JWT to obtain an access token. *These AEM Developer Console credentials are not deprecated and remain supported for AEM as a Cloud Service server-to-server access.*
- The OpenAPI examples in this guide use **OAuth Server-to-Server** via Adobe Developer Console, which is the recommended pattern for OpenAPI-based AEM APIs.

1.2.1 Open the AEM Developer Console

1. Log in to **Cloud Manager**: `https://experience.adobe.com/#/cloud-manager`
2. Select the **SHRSS AEM program**.
3. In the **Environments** section, find the target environment (e.g., `author-p135156-e1336227`).
4. Click the **… (ellipsis)** for that environment and choose **Developer Console**.
5. Log in if prompted; you should land in the **AEM Developer Console** for that environment.

Reference: "Developer Console access" and roles in Developer console.

1.2.2 Create a new technical account

1. In the AEM Developer Console, go to: **Tools → Integrations → Technical Accounts**.
2. Click **Create new technical account**.
3. Wait for the credentials page to load – it will show a JSON block containing:
    - `clientId`
    - `clientSecret`
    - `privateKey`
    - `certificate`
    - `email` (the technical account user, e.g. …`@techacct.adobe.com`)
    - `org` (IMS Org ID)
4. Click the **Download** icon and save the JSON file somewhere secure (this is the **service credentials JSON**).

This JSON is what your backend services will use to generate a JWT and exchange it for an **access token** to call AEM.

References: Generating Access Tokens for Server-Side APIs and Service credentials.

1.2.3 Treat the JSON as a secret

- Store the JSON in the team's standard **secret manager** (Azure Key Vault, AWS Secrets Manager, etc.).
- Never commit it to Git or share via email/Slack.

1.3 Configure AEM permissions for the technical account

By default, the technical account user is created in AEM with **Contributor/read** permissions only. You must grant it the right groups to read/write assets under `/content/dam`.

1. Log into the **AEM Author** UI for that environment as an AEM admin, e.g.: `https://author-p135156-e1336227.adobeaemcloud.com`
2. Go to **Tools → Security → Users**.
3. Search for the technical account's email shown in the service credentials JSON (for example: `8d4e6231-a621-4590-bbf5-61a27f8afebd@techacct.adobe.com`).
4. Open that user and go to the **Groups** tab.
5. Add the user to the appropriate groups, for example:

- `dam-users` (standard DAM access)
- Any other project-specific groups that grant read/write where needed under `/content/dam`.

6. Save.

Once this is done, any access token obtained from the service credentials will have those AEM permissions.

References: Service credentials and Sapphire-AEM: Steps for creating AEM Technical Account user.

1.4 Using the technical account programmatically (high level)

Your backend or integration will:

1. Read the **service credentials JSON**.
2. Generate a **JWT** signed with the provided `privateKey`.
3. Call Adobe IMS to exchange the JWT for an **access token**.
4. Call the AEM APIs with:

```
Authorization: Bearer <access_token>
```

For example, to hit the AEM Assets HTTP API:

```
GET https://author-p135156-
e1336227.adobeaemcloud.com/api/assets/shrss/hotel/en/photography/media/22795078_ImageLar
geWidth.jpg.json
```

Full code samples for this flow are available here:

- Generating Access Tokens for Server-Side APIs
- How to obtain access tokens using AEM-CS API client code sample

2 Code Samples

*SHRSS AEM Assets HTTP API cURL Examples for **/content/dam/shrss/corporate/photography***

2.1 Common variables

For brevity, assume these shell vars:

```
export AEM_HOST="https://author-p135156-e1336256.adobeaemcloud.com"
export AUTH="Authorization: Bearer <ACCESS_TOKEN>"
```

2.2 Traverse folders (R – list folder contents)

List the photography folder (JSON via Assets HTTP API):

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography.json" \
  -H "$AUTH"
```

- This returns a SIREN JSON document representing the folder and its **children** (assets and subfolders).

- Pattern: `/content/dam/...` → `/api/assets/...` (drop `/content/dam`) and add `.json`. See *Assets HTTP API* and *How to Update Your Content via AEM Assets APIs*.[1],)

2.3 Create a new folder under `photography` (C – folder)

Example folder name: `api-demo`

```
curl -X POST \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo" \
  -H "$AUTH" \
  -H "Content-Type: application/json" \
  -d '{
    "class": "assetFolder",
    "properties": {
      "title": "API Demo Folder"
    }
  }'
```

Verify:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo.json" \
  -H "$AUTH"
```

2.4 Create a new asset in that folder (C – file)

On **AEM as a Cloud Service**, binary upload via the Assets HTTP API is deprecated; use **Direct Binary Upload** instead.[2],)

You will create `sample.jpg` in:

```
/content/dam/shrss/corporate/photography/api-demo/sample.jpg
```

2.4.1 Step 1 – Initiate upload

```
curl -X POST \
  "$AEM_HOST/content/dam/shrss/corporate/photography/api-demo.createasset.html" \
  -H "$AUTH" \
  -H "Content-Type: application/json" \
  -d '{
    "fileName": "sample.jpg",
    "fileSize": 1234567,
```

```
    "mimeType": "image/jpeg"
  }'
```

- Response contains:
  - One or more `uploadURIs` (pre-signed URLs for binary PUT)
  - The `completeURI` callback
  - An `uploadToken`

Save the response as `initiate.json` for the next step.

2.4.2 Step 2 – Upload to blob store (PUT binary)

```
UPLOAD_URI=$(jq -r '.files[0].uploadURIs[0]' initiate.json)

curl -X PUT \
  "$UPLOAD_URI" \
  -H "Content-Type: image/jpeg" \
  --data-binary "@sample.jpg"
```

2.4.3 Step 3 – Complete upload

```
COMPLETE_URI=$(jq -r '.completeURI' initiate.json)
UPLOAD_TOKEN=$(jq -r '.uploadToken' initiate.json)

curl -X POST \
  "$AEM_HOST$COMPLETE_URI" \
  -H "$AUTH" \
  -H "Content-Type: application/json" \
  -d "{
    \"uploadToken\": \"$UPLOAD_TOKEN\"
  }"
```

If successful, the asset now exists at `/content/dam/shrss/corporate/photography/api-demo/sample.jpg`.

Verify via HTTP API:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg.json" \
  -H "$AUTH"
```

Direct Binary Upload is also available via the `aem-upload` Node.js library; see *Developer references for Assets – Asset upload* and *Direct Binary Access for external binary storage*.[2],)

2.5 Update asset metadata (U – asset metadata)

Example: update `dc:title` and `dc:description` on `sample.jpg`.

```
curl -X PATCH \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg" \
```

```
  -H "$AUTH" \
  -H "Content-Type: application/json" \
  -d '{
    "properties": {
      "dc:title": "Sample image uploaded via API",
      "dc:description": "Uploaded using Direct Binary Upload + Assets HTTP API"
    }
  }'
```

Confirm:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg.json" \
  -H "$AUTH"
```

Look for the updated properties under the asset's `properties` (metadata) section.

2.6 Delete asset (D – file)

```
curl -X DELETE \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg" \
  -H "$AUTH"
```

You can verify deletion by attempting a GET:

```
curl -X GET \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo/sample.jpg.json" \
  -H "$AUTH" -i
```

Expect a `404` if deletion succeeded.

2.7 Delete folder (D – folder)

Make sure the folder is empty (no assets/subfolders) or use the `?recursive=true` pattern as appropriate and acceptable for your use case.

Non-recursive delete:

```
curl -X DELETE \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo" \
  -H "$AUTH"
```

If you need recursive deletion (and your policies allow it):

```
curl -X DELETE \
  "$AEM_HOST/api/assets/shrss/corporate/photography/api-demo?recursive=true" \
  -H "$AUTH"
```

Be careful with `recursive=true` in shared folders.

3 Appendix A - Adobe Support ticket template (only if blocked)

Use this **only if** you cannot:

- see the **Developer Console** link,
- create a technical account, or
- see the **Integrations / Technical Accounts** section despite having the right roles.

3.1.1 Ticket subject

SHRSS – Enable AEM Developer Console technical account & service credentials for Assets APIs

3.1.2 Ticket body (copy/paste and fill in)

Product: AEM as a Cloud Service – Assets **Customer:** SHRSS **IMS Org ID:** `<ORG_ID>@AdobeOrg` **Cloud Manager Program:** `<Program Name> (Program ID: <programId>)` **Environment(s):** `author-p135156-e1336227` (and any others that are impacted)

Issue summary

We are trying to create an **AEM technical account** and **service credentials JSON** for programmatic access to the AEM Assets APIs on our AEM as a Cloud Service environments.

What we expect to do:

- As an IMS Org/System Admin and AEM admin, we should be able to:
    1. Log into Cloud Manager
    2. Open Cloud Manager → Program Overview → … → Developer Console for the SHRSS author environment
    3. Navigate to Tools → Integrations
- Actual result:
    - `<Describe exactly what you cannot see or do – missing menus, errors, etc.>`

Steps already taken

- Verified our IMS Org roles (System Admin or Org Admin)
- Verified AEM product profiles / admin access for Cloud Manager and the environment

Requested action from Support

- Verify SHRSS's entitlements and configuration for AEM Developer Console and technical accounts on this environment

- Enable or repair the **Developer Console → Integrations → Technical Accounts** capability so we can create service credentials

- Confirm any additional product profiles / roles needed for SHRSS to self-manage technical accounts going forward

This is required to unblock programmatic use of the AEM Assets APIs for SHRSS.

4 Appendix B - Optional: OpenAPI-based AEM Assets Author API (future-proof)

If you later want to use the **OpenAPI-based AEM Assets Author API** via **Adobe Developer Console** with OAuth Server-to-Server:

1. Make sure the environment is **modernized** and entitled for **AEM Assets APIs**.
2. In **Admin Console**, associate the **AEM Assets API Users** service with the relevant author product profile and add the architect as a **Developer** to that profile.
3. In **Adobe Developer Console**:
    - Create a project
    - **Add API → AEM Assets Author API**
    - Choose **Server-to-Server OAuth**
    - Select the product profile from step 2
4. Register the project's client ID in AEM via config pipeline (per the OpenAPI setup docs).

References: OpenAPI-Based APIs, Set up OpenAPI-based AEM APIs, and Invoke OpenAPI-based AEM APIs for server to server authentication.